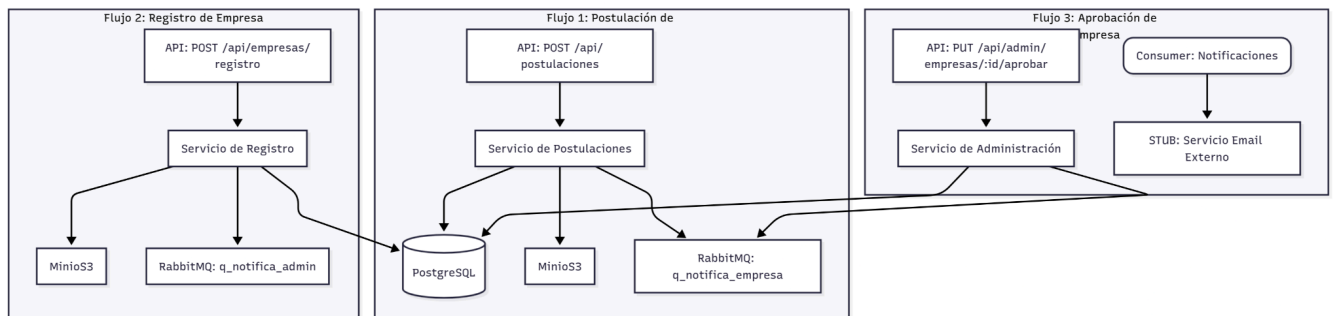


1) Selección de integraciones (mapa)



- **Servicio de Postulaciones (flujo 1)**

Valor de Negocio: Almacenar las ofertas de trabajo y las postulaciones de alumnos.

Riesgo: Si falla, se pierde la información esencial de la plataforma (ofertas de trabajo y postulantes).

Frecuencia: Alta. Cada vez que una empresa crea una oferta o un alumno postula.

- **Servicio de Registro (flujo 2)**

Valor de Negocio: Iniciar el proceso de aprobación de registro para empresas.

Riesgo: Si falla, el administrador nunca recibe la notificación para revisar a la nueva empresa, quedando el registro pendiente indefinidamente y obstaculizando el negocio.

Frecuencia: Cada vez que una empresa se registra.

- **Servicio de Administración en validación de empresas**

Valor de Negocio: Es el punto de control que valida la legitimidad de las empresas, protegiendo a los estudiantes de la UFRO de ofertas fraudulentas o inapropiadas.

Riesgo: Una falla aquí podría llevar a que una empresa malintencionada opere en la plataforma, además, una falla en la actualización de la BD implica que la empresa nunca avanzará en el sistema.

Frecuencia: Media. Ocurre cada vez que un administrador procesa una empresa (una vez por empresa).

2) Estrategia de integración

La estrategia de integración seleccionada para el proyecto es Híbrida, con una clara prioridad en el enfoque Bottom-Up para los componentes críticos de infraestructura.

Esta elección se justifica por la alta dependencia del sistema en la persistencia de datos (PostgreSQL y minio s3) y la entrega garantizada de mensajes asíncronos (RabbitMQ).

Si estos cimientos fallan, la funcionalidad de la API de alto nivel (Registro y Administración) es inviable.

Fases de las pruebas y su clasificación:

Fase	Módulos a Integrar y Probar	Estrategia utilizada	Justificación
Fase 1	PostgreSQL y Servicios de Almacenamiento (Minio S3).	Bottom-Up	Asegurar la persistencia de datos y archivos. Sin esto, ninguna otra funcionalidad puede operar.
Fase 2	Servicio de Mensajería Asíncrona: RabbitMQ Consumer y el STUB de Email Externo.	Bottom-Up	Garantizar la entrega de notificaciones críticas probando el eslabón más débil primero
Fase 3	Servicio de Registro (Flujo 2) Integración con la DB y la cola rabbitMQ	Top-Down	Se valida el primer flujo completo. La API llama a DB y aloja documentos en minio s3, realizando el flujo de arriba hacia abajo
Fase 4	Servicio de Administración (Flujo 3): Integración de aprobar empresa, con BD y la cola de rabbitMQ	Top-Down	Integración del punto de control crítico del negocio, verificando la autorización de la empresa y la activación del

			evento de notificación final realizando el flujo de arriba hacia abajo.
Fase 5	Servicio de Postulaciones (Flujo 1) Integrar con BD y la cola de rabbitMQ	Top-Down	Integración del flujo de alto tráfico, que requiere que las ofertas y las empresas estén estables y validadas realizando el flujo desde arriba hacia abajo.

3) Diseño de pruebas (plantilla)

Las pruebas están diseñadas para ejecutarse una vez que las fases de infraestructura Bottom-Up (Fase 1 y Fase 2) se han completado, centrándose en la integración de los flujos de negocio Top-Down (Fases 3, 4 y 5).

Flujo 2: Registro de Empresa:

ID / Título	IT-1 – Registro Exitoso de Empresa (Camino Feliz)
Integración	API Registro ↔ PostgreSQL ↔ MinioS3
Precondiciones/Fixture	1. No existe una empresa registrada con el email contacto@nuevaempresa.cl 2. El bucket documentos-legales de MinioS3 está disponible.
Entrada	HTTP POST /api/empresas/registro Payload: { "nombre": "Nueva Empresa", "email": "contacto@nuevaempresa.cl", ... } Archivo: rut_empresa.pdf
Pasos	1. Enviar la solicitud HTTP POST con el payload y el archivo adjunto.
Resultados Esperados	1. Respuesta 201 Created. 2. Nuevo registro en la tabla empresas con estado PENDIENTE. 3. El documento legal se almacena en MinioS3 con una URL registrada en la DB. 4. Un mensaje ingresa a la cola q_notifica_admin con el ID de la nueva empresa para iniciar el proceso de revisión.

ID / Título	IT-2 – Error: Registro con Email Duplicado (409)
Integración	API Registro ↔ PostgreSQL
Precondiciones/Fixture	1. Existe en la base de datos una empresa registrada con el correo contacto@nuevaempresa.cl

Entrada	HTTP POST /api/empresas/registro con correo duplicado.
Pasos	1. Enviar la solicitud HTTP POST.
Resultados Esperados	1. Respuesta 409 Conflict. 2. No se crea un nuevo registro. 3. MinioS3 no es invocado (el error ocurre antes de intentar guardar el archivo). 4. La cola q_notifica_admin permanece vacía.

Flujo 3: Aprobación de Empresa

ID / Título	IT-3 – Aprobación exitosa de empresa y notificación (Camino Feliz)
Integración	API Admin ↔ PostgreSQL ↔ RabbitMQ (q_notifica_empresa) ↔ Stub Email
Precondiciones/Fixture	1. Existe un administrador (id_admin_1). 2. Existe una empresa (id_empresa_1) con estado PENDIENTE. 3. WireMock (Stub Email) configurado para responder 200 OK ante el intento de envío de correo.
Entrada	HTTP PUT /api/admin/empresas/id_empresa_1/aprobar con JWT de administrador.
Pasos	1. Enviar la solicitud HTTP PUT a la API. 2. El Consumer escucha el mensaje de la cola y realiza el intento de envío de correo al stub.
Resultados Esperados	1. Respuesta 200 OK de la API de Administración. 2. La empresa cambia estado en BD a APROBADA. 3. La cola q_notifica_empresa recibe un mensaje. 4. Verifica la llamada al endpoint del Stub de correo con el payload de aprobación.

ID / Título	IT-4 – Error: Fallo Crítico en Servicio Externo de Email (Reintento/DLQ)
Integración	RabbitMQ (Consumer) ↔ Stub Email
Precondiciones/Fixture	1. Empresa en estado PENDIENTE en la BD 2. Stub Email configurado para devolver 503 Service Unavailable la primera vez. 3. La cola q_notifica_empresa recibe un mensaje de aprobación.
Entrada	Invocación asíncrona al Consumer de Notificaciones (no HTTP).
Pasos	1. El mensaje de la cola es consumido. 2. El Consumer intenta contactar al Stub de correo, que devuelve 503. 3. Se dispara la lógica de reintento de RabbitMQ.
Resultados Esperados	1. El mensaje fallido no es marcado como ACK y es enviado de vuelta a la cola, a un reintento programado, o a una Dead Letter Queue (DLQ). 2. WireMock registra la llamada fallida al servicio de email. 3. El estado de la empresa en BD es APROBADA (la API HTTP fue síncronamente exitosa)

Flujo 1: Postulación de Alumno

ID / Título	IT-5 – Postulación Exitosa (Camino Feliz)
Integración	API Postulaciones ↔ PostgreSQL ↔ MinioS3 ↔ RabbitMQ (q_notifica_empresa)
Precondiciones/Fixture	1. La BD contiene un alumno registrado (id_alumno_1). 2. Existe una empresa aprobada (id_empresa_1). 3. Existe una oferta laboral activa (id_oferta_1) publicada por id_empresa_1. 4. La cola q_notifica_empresa está vacía. 5. El bucket cvs de MinioS3 está disponible.

Entrada	HTTP POST /api/postulaciones Payload: { "id_oferta": "id_oferta_1" } Archivo: cv_alumno.pdf Token: JWT de id_alumno_1
Pasos	1. Enviar la solicitud HTTP POST a la API.
Resultados Esperados	1. Respuesta 201 creado con el ID de postulación. 2. Se registra una nueva entrada en la tabla postulaciones con estado recibida. 3. El archivo se almacena en el bucket cvs dentro de la carpeta correspondiente al alumno. 4. La cola q_notifica_empresa recibe un mensaje con datos de postulación y la URL del CV.

ID / Título	IT-6 – Error: Postulación a Oferta de Empresa NO APROBADA (403)
Integración	API Postulaciones ↔ PostgreSQL
Precondiciones/Fixture	1. La DB contiene un alumno válido (id_alumno_1). 2. Existe una empresa (id_empresa_1) con estado PENDIENTE o RECHAZADA. 3. Existe una oferta laboral activa (id_oferta_1) publicada por id_empresa_1.
Entrada	HTTP POST /api/postulaciones Payload: { "id_oferta": "id_oferta_999" } Token: JWT de id_alumno_1
Pasos	1. Enviar la solicitud HTTP POST.
Resultados Esperados	1. Respuesta 403 Forbidden (o 404 Not Found si el sistema oculta la existencia). 2. Ningún registro nuevo en la tabla postulaciones. 3. La cola q_notifica_empresa permanece sin mensajes.

4) Implementación

- **Framework de pruebas:** Jest, incluido por defecto en NestJS.
- **Cliente HTTP para pruebas de controladores:** Supertest, integrado con NestJS para la ejecución de solicitudes sobre la API.
- **Contenedores de infraestructura:** archivo `docker-compose.yml` destinado a levantar los servicios requeridos.
- **Simulación de servicios externos:** WireMock ejecutado en un contenedor Docker o, alternativamente, la librería `nock` si la comunicación HTTP se origina desde un *consumer* en RabbitMQ dentro del entorno Node.js.
- **Validación de efectos laterales:**
- **Base de Datos (PostgreSQL):** Utilización de TypeORM
- **Colas de mensajería (RabbitMQ):** Empleo de la librería `amqplib` con el objetivo de confirmar que el mensaje se publica correctamente o es dirigido a una *Dead Letter Queue* cuando corresponde.
- **Almacenamiento de archivos (MinioS3):** Verificación mediante la librería `@aws-sdk/client-s3`, compatible con Minio, garantizando la existencia del archivo en el *bucket* apropiado.

5) Evidencia reproducible

- Comandos para levantar entorno y ejecutar pruebas.
- Salidas de test (logs/resúmenes) y, cuando aplique, capturas de verificación de stub (p. ej., panel WireMock / `scope.isDone()` en Nock).
- Explicitar semilla o configuración que haga los tests deterministas.